Hitesh Rathore(HR) Notes

LINKEDIN LINK:
https://www.linkedin.com/in/hitesh-rathore-835279305/

HOW BROWSERS APPLY CSS EFFICIENTLY

The challenge: Apply CSS to thousands of elements fast without slowing down the page.

APPROACH 1 (NAIVE)

Render HTML first, then apply CSS

Read HTML line by line → Render on screen → Apply CSS afterwards → Re-render (again & again)

- Multiple re-renders
- More repaints
- More layout calculations
- Page loading becomes slower

Time Waste! (Page loading time increases)

APPROACH 2 (BETTER)

Store CSS first, then render HTML efficiently

Store / Parse all CSS → Read HTML line by line → Render with CSS (while reading) → Single render (Fast!)

How to find matching CSS for an element?

M-1: Linear Search	M-2: Use Map
Search in complete CSS	Store in Map / Hash
TC: O(n)	TC: O(1)

BUT... BIGGER PROBLEM!

CSS lookup is NOT simple hashmap lookup.

Why?

Because CSS selectors can be:

`div` `.container` `#id` `div p span` `.container > p` `input[type="text"]` `div:hover` `section article .card p`

- Hierarchical
- Context Dependent
- Dynamic
- Specificity Dependent

So browser cannot simply do:

`map["div"]`

☒

Because context, relationships and specificity matter!

IMPROVEMENT 1
GROUP CSS RULES SMARTLY

Instead of storing ALL CSS together, browser groups rules by selector type for faster matching.

Browser groups rules like:

.class	**Class Selectors**	.container .card .btn
#id	**ID Selectors**	#header #main #footer
div	**Tag Selectors**	div p span h1
[attr]	**Attribute Selectors**	input[type="text"] [href]
:hover	**Pseudo-class Selectors**	a:hover button:focus li:first-child
.a.b>p	**Complex (Combinator) Selectors**	.container p span section > .card p ul li a:hover .card .title + p

When browser sees an element, it checks ONLY relevant groups instead of all rules.

Huge Optimization!

BUT STILL A PROBLEM

What about selectors like:

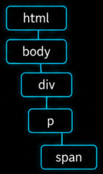
`.container p span`

This rule depends on:

Parent · Ancestors · Nesting

IMPROVEMENT 2
STORE RELATIONSHIPS

HTML is NOT flat. It has structure.
So browser creates a TREE.



Now browser can easily ask:
"Is this span inside p inside div?"

Much faster!

ANOTHER PROBLEM

Suppose 1000 elements use same class:

`.card { color: red; }`

Should browser calculate style 1000 times separately?

☒ Waste of CPU!

IMPROVEMENT 4
CACHE RESULTS

Browser can think:

"I already calculated this style once. So I will reuse it."

This becomes:
STYLE SHARING

Massive Optimization!

WHAT DOES "I ALREADY CALCULATED THIS STYLE" ACTUALLY MEAN?

Example HTML:

```html
<div class="card">A</div>
<div class="card">B</div>
<div class="card">C</div>
```

CSS:

```css
.card {
   color: red;
   width: 200px;
   margin: 10px;
   padding: 20px;
}
```

What browser does:

Browser computes FINAL STYLES.
Not just "class = card"

It figures out the COMPUTED STYLE for this element:

```
color       →  red
width       →  200px
margin      →  10px
padding     →  20px
display     →  block
font-size   →  16px
position    →  static
..          →  ..
```

} Computed Style

Why "Computed"?

Because browser may need to:

- ✓ Merge multiple CSS rules
- ✓ Resolve conflicts
- ✓ Apply inheritance
- ✓ Apply default browser styles
- ✓ Convert relative units
- ✓ Calculate percentages
- ✓ Handle media queries
- ✓ Handle specificity

Example:

```css
div {
  color: blue;
}

.card {
  color: red;
}
```

Final color = red ✓

This decision-making process is:
STYLE CALCULATION

SUMMARY OF OPTIMIZATIONS

1. Store CSS First (Approach 2) → 2. Group CSS Rules (Bucket by element) → 3. Build DOM Tree (Store relationships) → 4. Cache & Share Styles (Reuse computed style) → 5. Fast Matching (Context + Specificity) → Result: Fast Rendering & Smooth UX!

LIFE BEFORE CSS — THE DARK AGES OF THE WEB

To understand why CSS was invented, you need to feel the pain that developers were in before it existed.

1. HISTORICAL PROBLEM

The web was born in **1991**. Tim Berners-Lee created HTML for one purpose: sharing scientific documents at **CERN**. The idea was simple — mark up text with meaning:

```
<h1>  This is a heading </h1>
<p>   This is a paragraph </p>
<ul>  This is a list </ul>
```

> **HTML** was a semantic language. It described **what** content was, not **how** it should look.

Q) Why did Tim Berners-Lee create HTML?

Couldn't scientists just share papers in normal document formats at CERN?

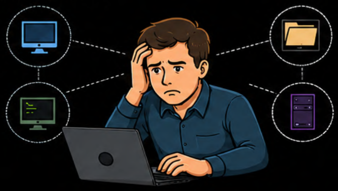
ANSWER:

The real problem was NOT creating documents. The real problem was CONNECTING information that was spread across different places.

At CERN, thousands of scientists were working on different experiments and research.

The problem was:

- Different computers
- Different operating systems
- Different file formats
- Different departments
- Different databases
- Information was scatered and isolated.



When a new scientist joined, it was difficult to find answers:

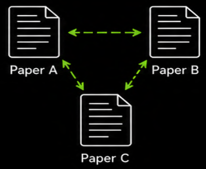
- Which document is where?
- Which experiment is related to which one?
- Which paper refers to which result?
- Who did what work?

Which document is where?	Which experiment is related to which?	Which paper refers to which result?	Who did what work?

FIRST PRINCIPLES PROBLEM

Problem was NOT: "How to make documents."

Problem was: "How to CONNECT knowledge."



Tim Berners-Lee realized:

Documents are isolated. But human knowledge is **CONNECTED.**

In the real world:

- One paper references another paper.
- One experiment depends on another experiment.
- One scientist builds on another scientist's theory.

So the idea came: "What if documents could **LINK** to each other?"

This idea is called: **HYPERTEXT**

And that became the World Wide Web.

WHY NORMAL DOCUMENTS FAILED

Normal documents are:

- Static
- Isolated
- Difficult to navigate

Example:

Suppose a paper says:

"See experiment #47 results"

Now the user has to manually:

- Search for the file
- Open the right folder
- Find the correct machine
- Then read it

Very painful.

"What if text itself could contain clickable connections?"

Meaning:

Instead of:	You could directly jump there.
See experiment #47	

This became:

HYPERTEXT

HyperText = text connected with links.

THEN HTML WAS CREATED

HTML was originally VERY simple.

- Not for apps.
- Not for animations.
- Not for beautiful websites.

Only for:

STRUCTURING DOCUMENTS

Example:

```
<h1>
<p>
<ul>
<a>
```

Notice something? These are SEMANTIC tags. Meaning-oriented.

HTML originally answered: "What is this content?"

NOT: "How should this look?"

HTML described the **structure** and **meaning**. → The styling problem came **MUCH** later.

THOUGHT QUESTIONS

QUESTION 1

If HTML already had <font>, bgcolor, size, etc.,
and it "worked" — why does mixing structure and style create such a serious engineering problem as a site grows? What breaks first?

MY ANSWER (Good intuition!)

If the style is the same for many elements, it has to be completely repeated in every single element.

YOUR ANSWER — AND DEEPER

You're right: repetition. You had to write the same style again and again. But why is repetition so dangerous in engineering?

It wasn't just a copy-paste problem.

Real scenario:
Your boss says: "Change all headings from red to blue."
You have 500 HTML pages. Each page has 8–10 headings like this:

```
<font face="Arial" size="5" color="#FF0000">Heading Text</font>
```

↓

- You have to change 5000+ places manually
- If you miss even one → one page looks different
- No tool could "find and replace across structure"
- Worse: if styles are written differently (#FF0000, red, #F00) → find-replace fails!

This is not just "inconvenience" — it's an architectural failure.

Old HTML approach:
Cost to change style
= O(pages × elements per page)
Small site → manageable
Large site → system collapses

CSS approach:
Cost to change style
= O(1)
Always one place.

A bigger problem you might not have thought of (yet)

<font> made content meaningless.
Consider this:

```
<font size="6" color="red">Breaking News</font>
```

Screen reader (blind user) →

What does it know?
- ✗ It's big
- ✗ It's red
- ✗ Maybe bold

→ But does it know…
- Is this a heading?
- Is this important?
- What's its meaning?

→ No idea.

With CSS and semantic HTML:

```
<h1>Breaking News</h1>
```

Screen readers, search engines, and tools know:
✓ This is a top-level heading (important, meaningful content).

LOOKS SIMPLE...
BUT IT'S DEEP!

Note: Repetition was the symptom. The real disease was coupling of style with structure, making both inseparable and hard to change.

QUESTION 2

CSS chose to be declarative ("color: red") rather than procedural ("set pixel at x,y to red").

What does the browser have to figure out to go from color: red to actual pixels on screen? What hidden complexity is the browser absorbing on your behalf?

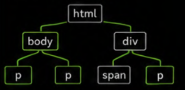
WHEN YOU WRITE color: red — THE BROWSER GETS TO WORK

1 Which element?
Selector Resolution

You write:

```
p { color: red; }
```

Browser has a DOM tree (thousands of nodes).
It must find every <p>.



Complex selectors:

```
div.container > p:first-child {
   color: red;
}
```

Browser must check for each <p>:
- Is parent div?
- Does it have class container?
- Is this the first child?
- Tree traversal + pattern matching across thousands of elements.

2 Which rule wins?
The Cascade

Many rules may apply to the same element.

```
p { color: red; }
.intro { color: blue; }
#main p { color: green; }
```

If <p class="intro"> inside #main → which color?

Browser runs cascade (priority system):
1. Importance (!important)
2. Origin (user > author > UA)
3. Specificity (ID > class > tag)
4. Source order (last wins)

This is why it's called "CASCADE".

3 Inherited values?
Inheritance Resolution

Some properties automatically flow from parent to child.

Browser must resolve inherited values for each property on each element.

Example:

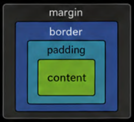
```
body { color: black; }
p { color: inherit; }
```

The <p> gets black (from body).

body (black)
↓
p (black)

4 How big is everything?
Layout (Reflow)

To render, browser must calculate sizes & positions of every element.
- Text measurement (font, size, line-height)
- Box model (margin, border, padding, width)
- Line breaks
- Wrapping
- Flexbox / Grid math



This is expensive and depends on many things.

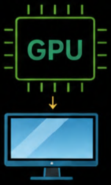
5 Where to paint?
Painting

Browser converts layout into paint commands.
- Backgrounds
- Text
- Borders
- Shadows
- Images
- Etc.

It decides the order (z-index, stacking contexts, opacity, clipping, etc.)

6 Finally: Pixels on screen
Rasterization (Compositing)

Painted layers → GPU takes over.



Millions of pixels are colored.

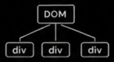
And it doesn't stop here...

JavaScript changes style at runtime

```
// Page load hone ke 3 second baad
setTimeout(() => {
  div.style.fontSize = "32px";
}, 3000);
```

→ Now the browser must:
- ✓ Recalculate style
- ✓ Re-run cascade
- ✓ Recompute layout (reflow)
- ✓ Repaint
- ✓ Composite again

→



Tree traversal again.
Work... again... again.

→

THE HIDDEN COMPLEXITY

By writing just color: red, you offload massive work to the browser.
That's the power of declarative CSS.

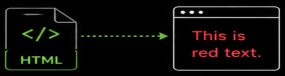
CSS TYPES

There are three ways to apply CSS to HTML.

1 INLINE CSS

Inline CSS is used to apply a unique style to a single HTML element.

How it looks:



Example Code:

```
<p style="color: red; font-size: 20px;">
  This is red text.
</p>
```

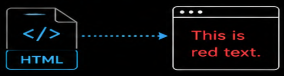
Output:

This is red text.

2 INTERNAL CSS

Internal CSS is used to define styles for a single HTML page inside the <style> tag.

How it looks:



Example Code:

```
<!DOCTYPE html>
<html>
  <head>
    <style>
      p { color: red; font-size: 20px; }
    </style>
  </head>
  <body>
    <p>This is red text.</p>
  </body>
</html>
```

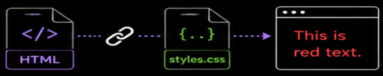
Output:

This is red text.

3 EXTERNAL CSS

External CSS is used to define styles in a separate .css file and link it to HTML.

How it looks:



HTML (example.html):

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet"
          href="styles.css">
  </head>
  <body>
    <p>This is red text.</p>
  </body>
</html>
```

CSS (styles.css):

```
p {
  color: red;
  font-size: 20px;
}
```

Output:

This is red text.

HTML + INLINE CSS — ACTUAL BROWSER INTERNAL WORKING

(Extreme Deep Dive)

```
<!DOCTYPE html>
<html>
<body>
    <h1 style="color: blue; font-size: 40px;">
        Hello
    </h1>
</body>
</html>
```

WHAT WE TRACE

From raw bytes → final pixels on screen **ONLY** for inline CSS.

NOT INCLUDED

- ✗ External CSS
- ✗ Internal CSS
- ✗ Any other CSS

PHASE 1 NETWORK RESPONSE ARRIVES

1 STEP 1 — Browser Receives Bytes

Server sends response.
Browser initially sees raw bytes.

```
00101001 01001011 10101000 00011110
00111001 01001000 01110100 01101101
01001100 00100000 01111001 01101111
01101111 01103100 00101110 00111010
...
```

At this stage browser has:

- ✗ No DOM
- ✗ No CSSOM
- ✗ No Render Tree
- ✗ No Layout
- ✗ Only bytes

PHASE 2 BYTE DECODING

2 STEP 2 — Encoding Detection

Browser checks (in order):

1. HTTP Headers (Content-Type)
2. `<meta charset>` in HTML
3. BOM (Byte Order Mark)

Suppose: **UTF-8** detected

→

3 STEP 3 — Byte Stream → Character Stream

Now browser converts bytes into characters.

```
<h1 style="color: blue;
  font-size: 40px;">
  Hello
</h1>
...
```

→

PRELOAD SCANNER (Runs in Parallel)

Quickly scans ahead for:

`<link>` `<script>` `<img>` `<video>` Fonts ...

Why?

To start network requests early and improve performance.

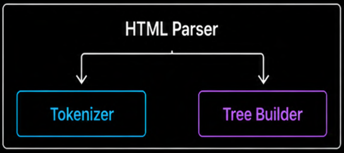
Does not block the main parser.

PHASE 3 HTML PARSER STARTS

IMPORTANT

HTML parser is not a single function.

Internally it has two major parts:



The Tokenizer reads the input and converts it into tokens.

The Tree Builder consumes those tokens and builds the DOM (Document Object Model).

4 STEP 4 — Tokenizer Starts

Tokenizer reads the input character-by-character.

Initially parser state:

DATA STATE

Meaning: Browser is reading normal text (not inside any tag).

INPUT STREAM (characters)

```
<h1 style="color: blue;
font-size: 40px;">Hello</h1>
<)
```

(Reading starts here)

5 STEP 5 — "<" Encountered

Browser sees: <

Tokenizer understands:

"Tag start ho sakta hai"

State changes:

DATA STATE → TAG OPEN STATE

INPUT STREAM

```
<h1 style="color: blue;
font-size: 40px;">Hello</h1>
^
```

("<" read)

6 STEP 6 — "h1" Read

Browser reads: h1

Tokenizer creates internally:

```
StartTagToken
  tagName = "h1"
```

Still:

⊗ No DOM node yet.

TOKENIZER STATE

TAG NAME STATE

Current Token (in progress)

```
StartTagToken {
  tagName: "h1"
}
```

(attributes: none yet)

7 STEP 7 — Attribute Parsing Starts

Browser encounters: style=

Tokenizer now enters:

ATTRIBUTE NAME STATE

INPUT STREAM

```
<h1 style="color: blue;
font-size: 40px;">Hello</h1>
^
```

(Reading attribute name)

8 STEP 8 — Style Attribute Value Read

Browser reads:

`"color: blue; font-size: 40px;"`

Tokenizer stores internally:

```
attributeName  = "style"
attributeValue = "color: blue;
                  font-size: 40px;"
```

TOKENIZER STATE

ATTRIBUTE VALUE (DOUBLE QUOTED) STATE

Current Token (in progress)

```
StartTagToken {
  tagName: "h1",
  attributes: {
    style: "color: blue;
       font-size: 40px;"
  }
}
```

IMPORTANT

✗ At THIS moment:

Browser still CSS understand nahi karta.

HTML tokenizer only knows:

"This is an attribute value string"

NOT:

- ⊗ color property
- ⊗ font-size
- ⊗ CSS declarations

9 STEP 9 — StartTagToken Completed

Tokenizer finally emits the completed token (to the Tree Builder).

Emitted Token:

```
StartTagToken {
  tagName: "h1",
  attributes: {
    style: "color: blue; font-size: 40px;"
  }
}
```

EMIT →

To Tree Builder

The Tree Builder will now process this token and build the DOM.

PHASE 4 — TREE BUILDER STARTS

The Tree Builder consumes tokens from the Tokenizer and constructs the DOM (Document Object Model) tree.

Tokens (from Tokenizer) → Tree Builder → DOM Tree (Hierarchy)

10 STEP 10 — Tree Builder Receives Token

Tree Builder receives the token emitted by the Tokenizer.

Its job:

Tokens (input) → DOM Hierarchy

Current Input Token:

```
StartTagToken {
  tagName: "h1",
  attributes: {
    style: "color: blue; font-size: 40px;"
  }
}
```

NOTE: Different browsers (Blink, WebKit, Gecko) may have different internal data structures for the Tree Builder, but the high-level algorithm is defined by the HTML Standard.

11 STEP 11 — DOM Node Creation

Tree Builder creates a DOM element node for `<h1>`.

- Browser (engine) allocates memory for the node.
- Conceptually, this is an HTMLHeadingElement.
- Internally, engines like Blink/WebKit use C++ objects (e.g., HTMLEeement / Element).

Conceptual Structure:

```
DOMNode (HTMLHeadingElement) {
  nodeType: ELEMENT_NODE (1),
  tagName: "H1",
  attributes: {},
  children: [],
  parent: null
}
```

Engine Note (May Vary):

- Blink: Element class (inherits from Node)
- WebKit: Element class
- Gecko: nsIContent / Element
- All engines follow the DOM Standard conceptually.

12 STEP 12 — Attributes Attached

Tree Builder attaches the attributes from the token to the created DOM node.

After this step, the node looks conceptually like:

```
DOMNode (HTMLHeadingElement) {
  nodeType: ELEMENT_NODE (1),
  tagName: "H1",
  attributes: {
    style: "color: blue; font-size: 40px;"
  },
  children: [],
  parent: null
}
```

★ IMPORTANT MOMENT

The browser now notices that the "style" attribute contains CSS syntax.
Therefore, the browser invokes the CSS Parser to process this inline style.

Inline Style Attribute (CSS Text) → CSS Parser → Style Declarations (Internal Representation)

IMPORTANT NOTES

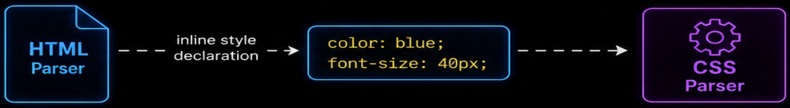
- The Tree Builder follows the HTML Standard algorithm.
- The exact internal representation (classes, objects, memory layout) differs across engines.
- DOM nodes are not yet styled or rendered at this point.
- CSS parsing happens after the attribute is attached but before style computation.
- Inline style has author origin and highest specificity (except !important from user/UA origin).

PHASE 5 — INLINE CSS PARSING

13 CSS PARSER STARTS

Browser sends inline style
`color: blue; font-size: 40px;`
to CSS parser subsystem.

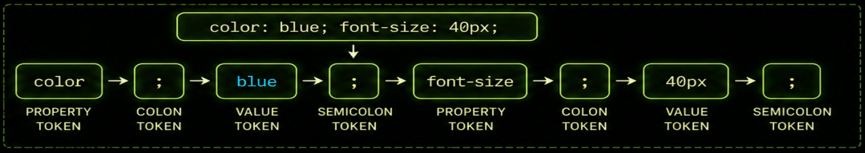
IMPORTANT
HTML parser ≠ CSS parser
Separate systems.



14 CSS TOKENIZATION

CSS parser breaks declaration into tokens.

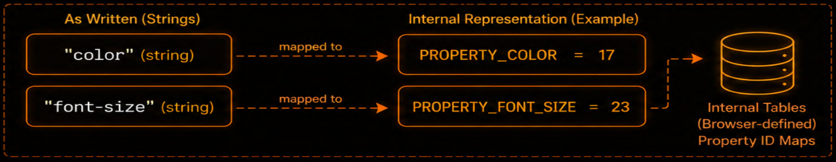
- ● color → PROPERTY TOKEN
- ● blue → VALUE TOKEN
- ● : ; → DELIMITER TOKEN (COLON / SEMICOLON)



15 INTERNAL CSS STRUCTURES

Browser does NOT keep everything as strings.
Instead, optimized internal representations are used.

Why?
Integer lookup MUCH faster than string comparison.



15 STEP 15 — INTERNAL CSS STRUCTURES (DEEP DIVE)

Browser does NOT keep CSS as raw strings. It converts everything into optimized internal representations.

1) INPUT RECEIVED

HTML Element

```
<h1 style="color: blue;
   font-size: 40px;">
```

↓

Style Declaration Sent to CSS Parser

```
color: blue;
font-size: 40px;
```

2) WHY NOT STORE STRINGS?

Example: “color” used 50,000 times

If stored as strings everywhere:

- Huge memory usage
- Slow string comparison
- Higher hashing cost
- CPU cache misses
- Expensive at scale

SOLUTION

Assign each property a unique ID (integer). Use IDs everywhere for fast lookup.

3) PROPERTY ID TABLE (Example)

ID (Integer)	Property Name
1	width
2	height
3	margin
4	padding
…	…
17	color
42	font-size
…	…

ⓘ IDs are implementation details and may vary from browser to browser.

4) PARSING RESULT (CONCEPTUAL)

Instead of storing strings:

```
{
  property: "color",
  value: "blue"
}
```

Browser creates internal representation:

```
{
  propertyID: 17,
  value: ...
}
```

(Value is also converted internally, see next panel)

5) VALUE OPTIMIZATION

`color: blue;`
Not stored as "blue".
Converted to numeric format.

`RGB(0, 0, 255)`

or

`0x0000FF`

`font-size: 40px;`
Not stored as "40px".
Converted to structured form.

```
value:  40
unit:   PX
```

6) FINAL INTERNAL REPRESENTATION (Conceptual)

After parsing:

```
color: blue;
font-size: 40px;
```

→

Property ID	Value Representation	Extra Info
17	RGB(0, 0, 255) (or 0x0000FF)	—
42	40 (numeric value)	Unit: PX

This internal structure is stored in style declarations for the element.

Internal Style Data

7) WHY INTEGER COMPARISON IS FASTER?

String Comparison (Slow)

`if (property == "color")`

CPU compares character by character: 'c' 'o' 'l' 'o' 'r'

More instructions, more time.

VS

Integer Comparison (Fast)

`if (propertyID == 17)`

Single integer comparison.

Much faster.

Critical at scale.

8) WHY STRUCTURED VALUES? (Example: font-size: 40px;)

If stored as raw string

`"40px"`

Problems:

- Math operations impossible
- Every time calculation needed, string must be parsed again.

Browser must parse again & again

`"40px"` → `40` + `px`

value unit

This repeated parsing is expensive.

Browser stores once, in structured form

```
value:  40
unit:   PX
```

Now math is easy:

40px + 10px → 50px

9) RENDERING NEEDS NUMERIC DATA

At paint/render stage, browser needs:

```
Red   = 0
Green = 0
Blue  = 255
```

(Numeric values) → "blue" (English string)

10) BIG PICTURE

Fast Lookup (Integer IDs) + Efficient Memory Usage + Faster Rendering Pipeline

NOTE Exact internal IDs, data structures, and optimizations are browser-specific and may change over time.

All major browsers follow similar principles:

Tokenization → Parsing → Internal Representation → Style Resolution → Rendering

Browsers (Examples)

Chrome Edge Firefox Safari

16 COLOR OPTIMIZATION

Instead of storing raw string

`"blue"`

browser may store:

`RGBA(0, 0, 255, 255)`
(4 bytes)

or

`Packed Integer: 0x0000FFFF`
(often 32-bit)

Browsers convert named colors to numeric values because the rendering engine (GPU) needs numeric color data.

(Exact format can vary by engine)

→

17 UNIT PARSING

Raw string

`"40px"`

Internally becomes:

```
value   =   40
unit    =   PX
```

NOT stored as raw string forever.

Value and unit are stored separately so that calculations (like calc(), inheritance, conversions) are easy and fast.
Unit enums (like PX, EM, REM, %) are typically stored as small integers.

→

18 INLINE STYLE DECLARATION OBJECT

Browser creates an internal declaration structure.
Conceptually:

```
InlineStyleDeclaration {
  color: RGBA(0, 0, 255, 255),
  font-size: {
    value: 40,
    unit: PX
  }
}
```

Actual internal structures and field names differ across engines (Blink, Gecko, WebKit).
This is a conceptual view.

→

19 STYLE ATTACHED TO DOM NODE

Now the DOM node internally contains a reference to this inline style declaration.

```
DOMNode (H1)
├ tagName: H1
├ attributes
└ inlineStyle
    ├ color: RGBA(0, 0, 255, 255)
    └ font-size:
        ├ value: 40
        └ unit: PX
```

IMPORTANT
Final style is NOT computed yet.
Browser still must apply:

- Default browser styles
- Inheritance
- Cascading rules (author stylesheets)

Q) If style is attached in DOM, will render tree not be created?

☒ NO

Attaching style to DOM node DOES NOT mean render tree is created.

DOM and Render Tree are fundamentally different.

1) WHAT IS DOM?

Represents document structure.

Example:

```
<h1 style="color: blue;">
  Hello
</h1>
```

DOM stores:

- Structure
- Relationships
- Attributes
- Content

DOM answers:
"What exists?"

2) WHAT IS RENDER TREE?

Represents visual (rendering) objects.
Created after style calculation.

Browser needs:

- Geometry (x, y, width, height)
- Layout information
- Paint instructions
- Z-order, stacking context, etc.

Render Tree answers:
"How should it visually appear?"

3) EXAMPLE DIFFERENCE

DOM (Structure)

```
html
 └─ body
     └─ h1
         └─ "Hello"
```

Render Tree (Visual Objects)

```
RenderBlock (H1)
• position: (100px, 50px)
• size: (120px × 48px)
• font-size: 40px
• color: blue
• paint order: ...
```

4) WHY SEPARATE?

DOM and Render Tree are separated for performance and scalability.

- DOM can contain non-visual nodes (script, meta, template, etc.)
- Rendering requires complex calculations
- Updates can be optimized independently

Modern browsers intentionally separate:
Document Logic
from
Visual Rendering Logic

5) KEY TAKEAWAY

HTML + Inline Style
↓
Parsed & Optimized (Steps 16–19)
↓
Attached to DOM Node
↓
Style NOT computed yet
↓
Style Resolution
↓
Render Tree Creation
↓
Layout → Paint → Composite

INLINE CSS: DOM, CSSOM, AND TREES – COMPLETE UNDERSTANDING

Q In inline CSS case:
Do HTML and CSS have separate trees?
Or are they stored in the same structure?

A

- ☑ DOM tree is always separate. (for HTML structure)
- ☒ For inline CSS, usually a separate full CSSOM tree (like internal/external CSS) is NOT created.
- ☑ Inline style data is parsed and attached directly to DOM elements as style declarations.
- ☑ Inline styles participate in the style system, cascade, and computed style generation.

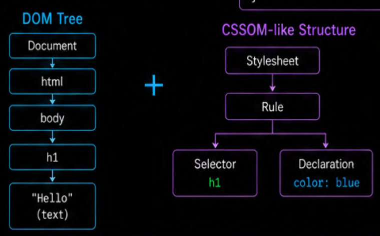
CASE 1 → INTERNAL / EXTERNAL CSS

Example HTML

```
<h1>Hello</h1>
```

Example CSS

```
h1 {
  color: blue;
}
```



How it works:
Browser matches CSS rules (h1) with DOM elements, then applies the styles.

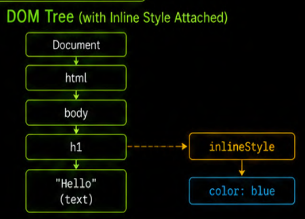
Selector matching is required.

CASE 2 → INLINE CSS

Example HTML

```
<h1 style="color: blue;">
  Hello
</h1>
```

No external CSS file or internal stylesheet needed.



How it works:

Style is already attached to the element.

No selector matching needed.

Directly used during style computation.

REAL DIFFERENCE

Type	How Browser Stores CSS
Internal / External CSS	Separate CSS Rule Structures (CSSOM-like tree) + Selector matching + Apply to matching elements
Inline CSS	Parsed style declarations attached directly to corresponding DOM elements + No selector tree needed + Direct association

DOES INLINE CSS CONTRIBUTE TO CSSOM?

- ☑ Yes, it contributes to the browser's style system.
- ☑ Yes, it participates in cascade.
- ☑ Yes, it affects computed styles.

☒ But it does NOT usually require a separate styleshent-rule tree like internal/external CSS.
(Because inline CSS is element-specific.)

MOST IMPORTANT MENTAL MODEL

DOM (What exists?)	≠	Render Tree (How it appears?)
• Structure • Relationships • Attributes • Content		• Layout (x, y, size) • Style (computed) • Geometry • Paint order • Visual instructions

Example:

```
<h1 style="color: blue;">
  Hello
</h1>
```

Example:

```
h1 at (100px, 50px)
color: blue, font-size: ...
paint order: ...
```

MOST TECHNICALLY ACCURATE FINAL ANSWER

- ☑ Proper DOM tree is created for HTML.
- ☑ Inline CSS is parsed and internal style/declaration objects are created.
- ☒ But usually, a separate stylesheet-rule-oriented CSSOM tree (like internal/external CSS) is NOT created.
- ☑ Instead, parsed inline style data is associated directly with corresponding DOM elements.

NOTE: ENGINE IMPLEMENTATION MAY VARY

Exact internal data structures, naming, and storage strategies vary across browser engines (Blink, Gecko, WebKit).

But the concepts and high-level architecture remain the same.

Blink Gecko WebKit

PHASE 6 — USER AGENT STYLESHEET

20 STEP 20 — BROWSER DEFAULT CSS

Even without writing CSS, browser already has default styles (User Agent Stylesheet).

Example (simplified):

```
h1 {
    display: block;
    font-size: 2em;
    margin-top: 0.67em;
    font-weight: bold;
}
```

User Agent Stylesheet

Built into the browser. Provides default styles for all HTML elements.

Different browsers may have slightly different defaults. (Values can vary across engines and versions.)

21 STEP 21 — STYLE RESOLUTION STARTS

Browser now combines styles from multiple sources.

User Agent Styles

```
h1 {
    display: block;
    font-size: 2em;
    margin-top: 0.67em;
    font-weight: bold;
    color: black;
}
```

+

Inline Styles

```
h1 style="color: blue;
 font-size: 40px;"
```

(already attached to this element)

↓

Next → Cascade Resolution
(Determine final values)

22 STEP 22 — CASCADE RESOLUTION

Example:

User Agent (Low Specificity)

```
h1 {
  color: black;
}
```

+

Inline Style (Very High Specificity)

```
style="color: blue"
```

→

Final Value

```
color: blue
(wins)
```

Why?

Inline styles have higher specificity than user agent styles.

IMPORTANT OPTIMIZATION

Inline CSS requires NO selector matching, because it is already directly attached to the element. Huge optimization!

PHASE 7 — COMPUTED STYLE CREATION

23 STEP 23 — COMPUTED STYLE OBJECT

Browser creates the final resolved style object (Computed Style) for the element.

Conceptual ComputedStyle:

```
ComputedStyle {
  color: rgb(0, 0, 255),
  fontSize: 40px,
  display: block,
  fontWeight: bold,
  marginTop: 0.67em,
  /* ... many more properties */
}
```

What is resolved here?

- ✓ Inheritance resolved
- ✓ Default values applied
- ✓ Units converted / finalized
- ✓ Cascade resolved
- ✓ Relative values computed (as far as possible)

Computed style is per element and cached for performance. (Actual implementations vary: Blink, Gecko, WebKit.)

PHASE 8 — RENDER TREE CREATION

24 STEP 24 — RENDER OBJECT CREATION

Browser creates visual rendering objects (Render Objects) from DOM + Computed Style.

DOM Tree

Document
↓
html
↓
body
↓
h1
↓
"Hello"
(text)

Render Tree (Visual Objects)

RenderBlock (for h1)
- display: block
- color: rgb(0, 0, 255)
- font-size: 40px
- font-weight: bold
- ... (other computed styles)

IMPORTANT

DOM ≠ Render Tree

Render Tree contains ONLY objects that generate visual output.

Non-visual nodes (like `<script>`, `<meta>`, etc.) do NOT appear in Render Tree.

25 STEP 25 — TEXT NODE RENDER OBJECT

Browser also creates render object for the text content.

From DOM text node:

"Hello"
(text)
↓
RenderText
("Hello")

Text is laid out and painted through its own render object.

KEY TAKEAWAYS

- 23 Computed Style = Final resolved style for the element after cascade, inheritance, defaults etc.
- 24 Render Objects are created from DOM elements that produce visual output (Render Tree).
- 25 Text nodes also get their own render objects (RenderText).
- **Note:** Exact internal names (RenderBlock, RenderText, etc.) and structures vary across browsers (Blink, Gecko, WebKit), but the overall process is the same.

PHASE 9 — LAYOUT

26 STEP 26 — GEOMETRY CALCULATION

Browser computes geometry for each render object.

Layout Calculations Include:

- ✓ Width
- ✓ Height
- ✓ Position (x, y)
- ✓ Line box metrics
- ✓ Baseline
- ✓ Margins / Padding / Borders
- ✓ Inline formatting context
- ✓ ... and more

Example Impact of Inline Style:

- `font-size: 40px`

↓

Affects:

- Text metrics (glyph size)
- Line height
- Element height
- Line wrapping
- Baselines
- Layout dimensions

Render Tree Before Layout

RenderBlock (h1)
↓
RenderText ("Hello")

→

Render Tree After Layout

RenderBlock (h1)
- x: 8px
- y: 16px
- width: 312px
- height: 56px

RenderText ("Hello")
- x: 8px
- y: 24px
- width: 104px
- height: 40px

Exact values depend on many factors:

- Viewport width
- Parent styles
- Fonts
- Device pixel ratio
- Browser engine
- Writing mode
- ...

FLOW SUMMARY (Steps 20 → 26)

20 User Agent Styles (Browser Defaults)
↓
21 Combine with Inline Styles
↓
22 Cascade Resolution (Inline wins)
↓
23 Computed Style Object Created
↓
24 Render Object (RenderBlock) Created
↓
25 Text Render Object (RenderText) Created
↓
26 Layout (Geometry Calculation)

Result:
Everything is now ready for the next phase → Paint (rasterization and drawing pixels to the screen).

IMPORTANT NOTES

- Internal object names (RenderBlock, RenderText, etc.) vary across browsers.
- The overall pipeline (style resolution → computed style → layout) is standardized by web specifications.
- Implementations differ in data structures, caching, invalidation, and optimizations.
- Inline styles skip selector matching, making them very fast to apply.

Browsers (Examples)

Chrome Edge Firefox Safari

Q) Were all geometry calculations already done during Render Tree creation?

NO.

Render Tree creation and Layout calculation are two different internal browser phases.

The browser intentionally separates them for performance optimization.

Technically Accurate Answer

- ☑ Render Tree creation does NOT fully calculate geometry.
- ☑ Render Tree mainly represents visual/rendering relationships.
- ☑ Exact positions and dimensions are computed later during the Layout phase.
- ☑ Browser separates these phases because layout calculation is computationally expensive.
- ☑ This separation allows browsers to avoid unnecessary layout work when only non-geometric properties change.

1) RENDER TREE CREATION

Purpose: Determine "What visual objects should exist?"

Browser creates visual/render objects from:

DOM + Computed Styles

Conceptually:

```
RenderBlock (h1)
└── RenderText ("Hello")
```

At this point, browser already knows:

- ☑ element is visible
- ☑ text should render
- ☑ computed color is blue
- ☑ font-size is 40px
- ☑ display is block

BUT browser still may NOT know:

- ☒ exact x position
- ☒ exact y position
- ☒ exact width
- ☒ exact height
- ☒ line wrapping result
- ☒ exact baselines

Meaning: "This should visually exist."

2) LAYOUT PHASE

Purpose: Determine "Exactly where and how large should everything be rendered?"

NOW browser computes physical geometry.
This stage is called: Layout / Reflow / Formatting
(engine terminology may vary)

Example Result:

```
RenderBlock (h1)
├── x = 20px
├── y = 100px
├── width = 180px
└── height = 52px
```

Layout calculates:

- ☑ x position
- ☑ y position
- ☑ width
- ☑ height
- ☑ line boxes
- ☑ baselines
- ☑ overflow areas
- ☑ ...

Meaning: "Exactly where and how large?"

INTERNAL PIPELINE (Conceptually)

STEP A
Render Tree Creation

Browser creates render objects.

```
RenderBlock (h1)
└── RenderText ("Hello")
```

Meaning: "This should visually exist."

→

STEP B
Layout

Browser computes:

- Where exactly?
- How large?
- How much space?

Example Result:

```
RenderBlock (h1)
  x = 20
  y = 100
  width = 180
  height = 52
```

→

STEP C
Paint

Browser paints pixels on the screen.

(Color, text, borders, shadows, etc.)

→

STEP D
Composite

Browser composites layers and shows final result.

(Final pixels on screen)

WHY BROWSERS SEPARATE THESE PHASES?

Suppose only color changes:

`element.style.color = "red";`

→ Now browser usually needs:

- ☑ style recalculation
- ☑ repaint

→ **BUT:**

- ☒ layout NOT needed

Because geometry did not change.

If browser mixed everything together, every small style change would trigger expensive layout calculations unnecessarily. That would be extremely slow.

STEP C vs STEP D — PAINT vs COMPOSITE (Actual Difference Clearly)

STEP C — PAINT

Purpose: Browser converts each render object into actual pixels, It decides exactly HOW each element should look.

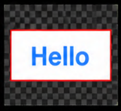
Input: Layout / Render Tree

```
RenderBlock (h1)
      |
RenderText ("Hello")
```

What browser does in Paint phase:

- Draw background (colors, images, gradients)
- Draw text (fonts, color, decoration)
- Draw borders (style, width, color)
- Draw shadows
- Draw outlines
- Apply anti-aliasing
- Resolve visibility, clipping, effects

Output of Paint:
Pixel content (per layer)



This is a bitmap (pixels)
Painted into a layer.
(Still not on screen yet)

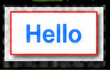
Paint Think Like:

"Screen par kya draw karna hai?"
(Drawing pixels for each element)

Example Style

```
color: blue;
background: white;
border: 2px solid red;
text-shadow: 2px 2px 4px rgba(0,0,0,0,3);
font-size: 40px;
```

Paint Result (Pixels)



STEP D — COMPOSITE

Purpose: Browser takes already painted layers and combines (composites) them in the correct order to produce the final frame on screen.

Input: Multiple Painted Layers

1. Background Layer (painted)
2. Text Layer (Hello) (painted)
3. Border Layer (painted)
4. Shadow Layer (painted)
5. Other Layers (e.g. navbar, ads, etc.)

...

What browser does in Composite phase:

- Determine stacking order (z-index)
- Position / move layers (using transform, scroll, etc.)
- Apply opacity, blending
- Clip layers
- Use GPU to combine layers efficiently
- Produce final frame

Output of Composite: Final Frame on Screen

This is what user finally sees.
(Shown on screen)

Composite Think Like:

"Already painted cheezo ko kaise combine karke final screen par dikhana hai?"
(Combining layers, not drawing pixels)

Example Scenario

If we change only transform: `translateX(100px);`

Browser often:

- ☑ Does NOT repaint
- ☑ Does NOT do layout

→ Only composite phase me layer move karega.
→ Fast & Smooth (GPU)

PAINT vs COMPOSITE — KEY DIFFERENCE

Aspect	PAINT (Step C)	COMPOSITE (Step D)
Main Work	Pixels generate karna (drawing content)	Layers combine karna (assembly)
Input	Layout info + Render objects	Already painted layers
Output	Bitmap (pixels) per layer	Final frame on screen
Examples	Draw text, background, border, shadow	Draw text, apply opacıd, barder, shadow
Expensive?	Relatively expensive (pixel drawing)	Usually cheaper (GPU optimized)

REAL PIPELINE (Simplified)

DOM + CSS → Style Calculation → Render Tree → Layout (Reflow) → Paint (Draw Pixels) → Composite (Combine Layers) → Screen (Output)

Paint: Creates pixels

Composite: Combines layers

Quick Analogy

PAINT = Artist Drawing
Har cheez ko canvas par draw karta hai.

COMPOSITE = Video Editor
Already bani clips/layers ko sahi order me jod ke final video banata hai.

INLINE CSS (style="...")

ADVANTAGES

✓ **No Network Request**
Inline styles are part of HTML, so no additional HTTP request is made.

✓ **No Selector Matching**
Browser does not need to match any CSS selector. The style is already directly applied to the element.

✓ **Immediate Availability**
Styles are applied as soon as the HTML is parsed. No dependency on external or internal stylesheets.

EXAMPLE

```
<h1 style="color: red; font-size: 24px;">Hello</h1>
```

Inline CSS

DISADVANTAGES

☒ **Poor Caching**
Same styles repeated in many elements cannot be cached effectively by the browser.

☒ **Style Sharing Harder**
Cannot be easily shared across multiple elements or pages.

☒ **Reusability Poor**
No reusability. Every time the style is needed, it must be written again.

☒ **Larger HTML Size**
HTML becomes bulky and increases the size of the document.

⚠ **BIGGEST DISADVANTAGE**

Maintainability Issues
Hard to maintain when the project grows. Any change requires editing many places, increasing the chance of errors.

Since inline CSS is fast, do we write inline in Tailwind CSS?

SHORT ANSWER

- ☒ Tailwind CSS is not actual inline CSS.
- ✓ Tailwind mostly uses normal CSS classes.
- ☒ Thus browser behavior is not like inline CSS.
- ✓ Tailwind generally follows external/internal stylesheet model.

KEY POINT

Tailwind = Utility-first CSS framework but still stylesheet-based (class-based), not inline CSS.

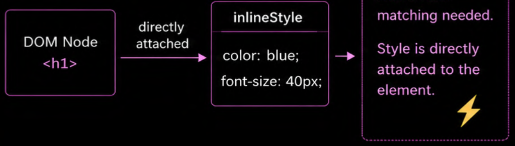
INLINE CSS vs TAILWIND CSS UNDERSTANDING

REAL INLINE CSS (Actual Inline)

HTML

```
<h1 style="color: blue; font-size: 40px;">Hello</h1>
```

Browser Internally


DOM Node
<h1>
directly attached
inlineStyle
color: blue;
font-size: 40px;
No selector matching needed.
Style is directly attached to the element.

TAILWIND CSS (Not Inline)

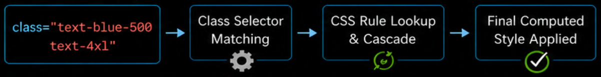
HTML

```
<h1 class="text-blue-500 text-4xl">Hello</h1>
```

Tailwind Generates CSS (Build Time)

```
.text-blue-500 { color: #3b82f6; }
.text-4xl { font-size: 2.25rem; }
```

Browser Internally


class="text-blue-500 text-4xl"
Class Selector Matching
CSS Rule Lookup & Cascade
Final Computed Style Applied

Still class-based CSS. Browser does selector matching, rule lookup, cascade resolution.

BUT TAILWIND FASTER FEEL KYU HOTA HAI?

Because Tailwind CSS selectors mostly **single-class selectors** hote hain.

Examples:

`.text-red-500` `.flex` `.mt-4` `.p-4` `.bg-blue-500`

These are VERY cheap selectors.

Browser Ko Matching Easy Ho Jati Hai
Browsers are highly optimized for class lookup using internal data structures (hash maps, bloom filters, etc.).

COMPARE

EXPENSIVE CSS (Deep/Complex Selector)

```
.sidebar .menu ul li a span.title
```

TAILWIND CSS (Simple Class Selector)

```
.text-blue-500
```

Complex ancestor traversal needed. Slow matching.

Simple class lookup only. Fast matching.

Result: Tailwind selectors are much faster to match than deeply nested traditional CSS.

INLINE CSS vs TAILWIND CSS IN REAL WORLD

INLINE CSS (Theoretically Fastest Matching)

✦ No selector matching at all.
✦ Styles are directly attached to the element.
✦ Browser applies immediately.

Example:

```
<button style="background: blue; padding: 16px;
color: white; border-radius: 8px;">Click</button>
```

BUT Problems:

☒ Repeated styles in HTML
☒ Poor caching
☒ Style sharing harder
☒ Reusability poor
☒ Larger HTML size

BIGGEST DISADVANTAGE

Maintainability Issues

Hard to maintain when the project grows. Any change requires editing many places, increasing the chance of errors.

TAILWIND CSS (Class-Based, Optimized)

☑ Uses simple, single-class selectors.
☑ Very cheap to match for browsers.
☑ Highly reusable across the project.
☑ Better caching & style sharing.
☑ Keeps HTML smaller than inline CSS.

Example:

```
<button class="bg-blue-500 p-4 text-white
rounded">Click</button>
```

Benefits:

☑ Simple selectors = fast matching
☑ CSS rules are reusable
☑ Browser can cache aggressively
☑ Computed style sharing possible
☑ Better maintainability

REALITY CHECK

Even though inline CSS avoids selector matching completely, in real projects performance difference is usually tiny because modern browsers:

- Class selectors are extremely optimized
- Use caching
- Use style sharing
- Optimize invalidation
- GPU acceleration helps in rendering

BOTTOM LINE

Tailwind is not inline CSS, but its utility-class approach makes it very close to optimal in real-world performance.